



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Batch: B2 Roll No.: 16010124107

Experiment No.9

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of the Staff In-charge with date

Objective: To learn the

Title: Study, Implementation and Analysis of 8-Queens problem.

Backtracking strategy of problem solving for 8-Queens problem

CO to be achieved:

Sr. No	Objective
CO 1	Compare and demonstrate the efficiency of algorithms using asymptotic complexity notations.
CO 2	Analyze and solve problems for divide and conquer strategy, greedy method, dynamic programming approach and backtracking and branch & bound policies.

Books/ Journals/ Websites referred:

1. Ellis horowitz, Sarataj Sahni, S.Rajsekaran,” Fundamentals of computer algorithm”, University Press
2. T.H.Cormen ,C.E.Leiserson,R.L.Rivest and C.Stein,” Introduction to algorithms”,2nd Edition ,MIT press/McGraw Hill,2001
3. <http://www.math.utah.edu/~alfeld/queens/queens.html>
4. <http://www-isl.ece.arizona.edu/ece175/assignments275/assignment4a/Solving%208%20queen%20problem.pdf>
5. http://www.slideshare.net/Tech_MX/8-queens-problem-using-back-tracking
6. <http://www.mathcs.emory.edu/~cheung/Courses/170.2010/Syllabus/Backtracking/8queens.html>
7. <http://www.geeksforgeeks.org/backtracking-set-3-n-queen-problem/>
8. <http://www.hbmeyer.de/backtrack/achtdamen/eight.htm>

Pre Lab/ Prior Concepts:

Data structures, Concepts of algorithm analysis



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Historical Profile:

The **N-Queens puzzle** is the problem of placing N queens on an $N \times N$ chessboard so that no two queens attack each other. Thus, a solution requires that no two queens share the same row, column, or diagonal.

New Concepts to be learned:

Application of algorithmic design strategy to any problem, Backtracking method of problem-solving Vs other methods of problem solving, 8- Queens problem and its applications.

Algorithm N Queens Problem: -

```
void NQueens(int k, int n)
// Using backtracking, this procedure prints all possible placements of n queens on an n X n
chessboard so that they are nonattacking.
{
    for (int i=1; i<=n; i++)
    {
        if (Place(k, i))
        {
            x[k] = i;
            if (k==n)
                for (int j=1; j<=n; j++)        Print x[j] ;
            else NQueens(k+1, n);
        }
    }
}
```

```
Boolean Place(int k, int i)
// Returns true if a queen can be placed in  $k^{\text{th}}$  row and  $i^{\text{th}}$  column. Otherwise it returns false.
// x[] is a global array whose first (k-1) values have been set. abs(r) returns absolute value of
r.
{
    for (int j=1; j < k; j++)
        if ((x[j] == i) // Two in the same column
            || (abs(x[j]-i) == abs(j-k))) // or in the same diagonal
            return(false);
    return(true);
}
```



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

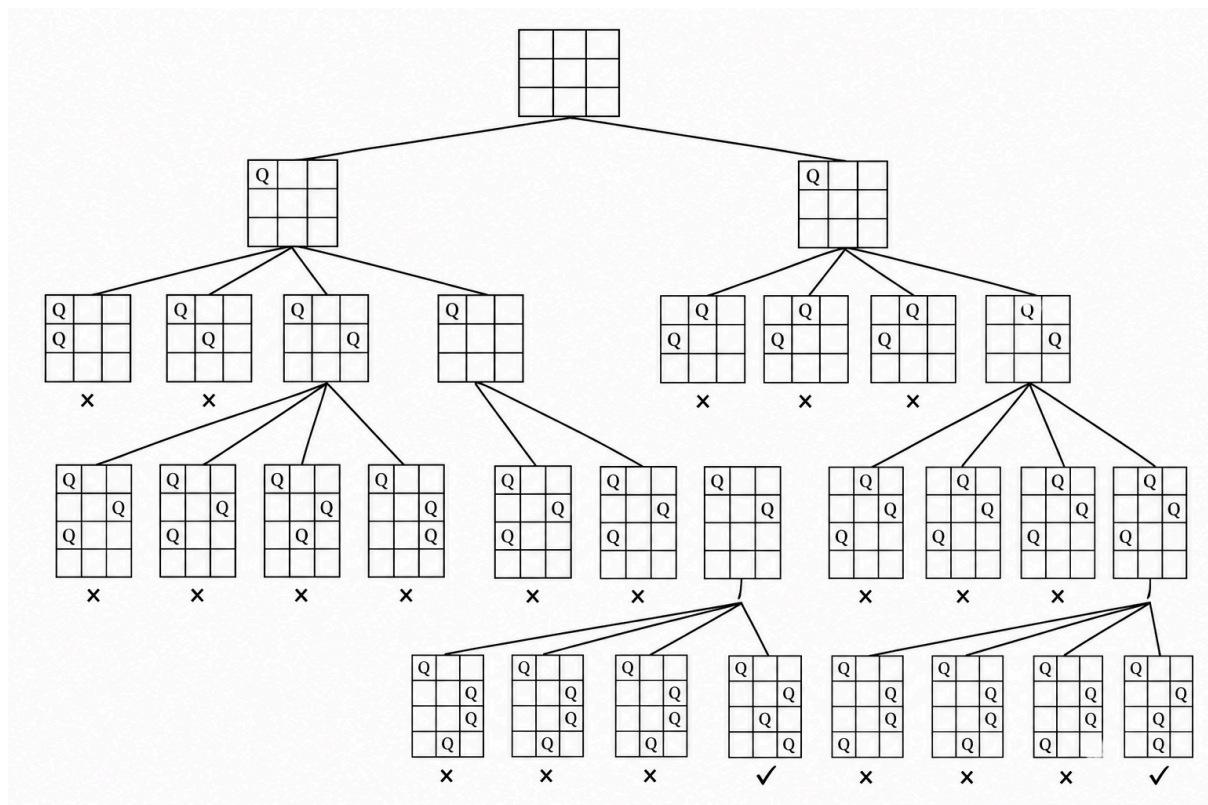
Example 8-Queens Problem:

The eight queens puzzle is the problem of placing eight chess queens on an 8×8 chessboard so that no two queens threaten each other i.e. no two queens share the same row, column, or diagonal.

Solution Using Backtracking Approach:

The idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for clashes with already placed queens. In the current column, if we find a row for which there is no clash, we mark this row and column as part of the solution. If we do not find such a row due to clashes then we backtrack and return false.

State Space tree for N-Queens (Solution):



Implementation (Code):

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define yes cout << "YES" << endl;
#define no cout << "NO" << endl;
#define input for(int &i:v) cin >> i;
#define vall v.begin(),v.end()
```



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

```
void print(const vector<int>& v) {
    for (int i : v) cout << i << " ";
    cout << endl;
}

void printBoard(vector<int>& queens, int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            cout << (queens[i] == j ? "Q" : ".");
        cout << "\n";
    }
    cout << "\n";
}

bool isSafe(vector<int>& queens, int row, int col) {
    for (int i = 0; i < row; i++) {
        if (queens[i] == col) return false;
        if (abs(queens[i] - col) == abs(i - row)) return false;
    }
    return true;
}

void solve(int row, int n, vector<int>& queens, int& count) {
    if (row == n) {
        count++;
        cout << "Solution " << count << ":\n";
        printBoard(queens, n);
        return;
    }
    for (int col = 0; col < n; col++) {
        if (isSafe(queens, row, col)) {
            queens[row] = col;
            solve(row + 1, n, queens, count);
            queens[row] = -1;
        }
    }
}

void solve() {
    int n;
```



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

```
cin >> n;

vector<int> queens(n, -1);
int count = 0;
solve(0, n, queens, count);

cout << "Total solutions: " << count << "\n";
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    solve();
}
```

OUTPUT:

```
4
Solution 1:
.Q..
...Q
Q...
..Q.

Solution 2:
..Q.
Q...
...Q
.Q..

Total solutions: 2
PS C:\Users\syeda\Music\SYN
```

Algorithm:

1. Start
2. Read n
3. Create array queens of size n
Initialize all values as -1
4. Set count = 0
5. Call recursive function solve(row = 0)
6. In solve(row):



K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

- a. If row equals n
 Increase count
 Print board configuration
 Return
- b. For each column from 0 to n-1
 - i. Check if placing queen is safe
 - ii. If safe
 Place queen at $\text{queens}[\text{row}] = \text{col}$
 Call $\text{solve}(\text{row} + 1)$
 Remove queen by setting $\text{queens}[\text{row}] = -1$
- c. End for
- d. End solve
7. After recursion ends
 Print total count of solutions
8. Stop

Analysis of Backtracking solution:

The backtracking solution for the N-Queens problem explores all possible ways of placing queens on the board one row at a time. At each row, it tries every column and checks whether placing a queen is safe based on column and diagonal constraints. If a safe position is found, the algorithm places the queen and moves to the next row. If no valid position is available, it backtracks to the previous row and tries a different column.

The state space of the problem can be represented as a tree where each level corresponds to a row and each node represents a column choice. In the worst case, the algorithm explores all possible configurations, leading to a time complexity of $O(n!)$. This is because for the first row there are n choices, for the second row $n-1$ choices, and so on, although many branches are pruned early due to safety checks.

Backtracking improves efficiency by eliminating invalid placements as soon as conflicts are detected, which reduces the number of nodes explored in practice. However, the worst-case time complexity remains exponential. The space complexity is $O(n)$ due to the recursion stack and the array used to store queen positions.

CONCLUSION:

In conclusion, the backtracking approach is a complete and systematic method that guarantees finding all valid solutions. It is efficient for small values of n but becomes computationally expensive for larger boards due to the rapid growth of possible configurations.